



UNIVERSIDAD
DE SALAMANCA
CAMPUS DE EXCELENCIA INTERNACIONAL



800 AÑOS
1218 - 2018

MÁSTER UNIVERSITARIO

GEOTECNOLOGÍAS CARTOGRÁFICAS



UNIVERSIDAD DE SALAMANCA
Campus de Ávila
CAMPUS DE EXCELENCIA INTERNACIONAL



Contenido

1	Herramientas para trabajar con SIG.....	2
1.1	Descargar e instalar QGIS.	2
1.2	Crear cuenta en PyCharm.....	5
1.3	Descargar PyCharm.....	7
1.4	Instalar PyCharm.....	8
2	Comenzar a programar en Python.....	14
2.1	Programas lineales.	14
2.1.1	Programa Área del círculo.	17
2.2	Programas condicionales.	17
2.3	Bucle For.	18
2.4	Bucle While.	18
2.5	Funciones en Python.....	19
2.6	Clases en Python.....	21
3	Desarrollo de complementos para QGIS	25
3.1	Configuración de QGIS.	25
3.2	Instalar plugins.	26
3.3	Crear el primer plugin básico.	31
3.4	Personalizar la interfaz gráfica del Plugin.	39
3.5	Modificar código del Plugin.	43
3.6	Depuración del Plugin.	44
4	Operaciones habituales en plugins de QGIS.....	50
4.1	Crear Plugin con lectura de archivo CSV y carga de datos.....	50
4.1.1	Modificar el diseño del plugin	53
4.1.2	Asignar código al plugin	56

4 Operaciones habituales en plugins de QGIS

4.1 Crear Plugin con lectura de archivo CSV y carga de datos.

Queremos crear un complemento que lea los datos de ubicación de paradas de autobús urbano en Madrid. Para probar utilizaremos los datos de 20 paradas. Los datos tiene el aspecto:

```
1 stop_id;stop_code;stop_name;stop_desc;stop_lat;stop_lon;zone_id;stop_url;location_type;parent station;stop_timezone;wheelchair_board
2 par_8_06686;06686;AV.BURGOS-C.C.SANCHINARRO;Avda de Burgos 133;40.4980545043945;-3.66299557685852;A;http://www.crtm.es;0;;Europe/Mac
3 par_8_06687;06687;AV.BURGOS-DOMINICOS;Avda de Burgos 11300;40.5014419555664;-3.65994882583618;A;http://www.crtm.es;0;;Europe/Madrid;
4 par_8_06689;06689;CTRA.AL-CUESTA BLANCA;Ctra de Irún 13700;40.5217704772949;-3.65101075172424;B1;http://www.crtm.es;0;;Europe/Madrid;
5 par_8_06690;06690;CTRA.IRÚN-CONCESIONARIO;Ctra de Irún 14700;40.5337944030762;-3.64185833930969;B1;http://www.crtm.es;0;;Europe/Madrid;
6 par_8_06691;06691;AV.OLÍMPICA-C.C.LA VEGA;Avda Olímpica 9;40.5352478027344;-3.63698363304138;B1;http://www.crtm.es;0;;Europe/Madrid;
7 par_8_06692;06692;AV.OLÍMPICA-PABELLÓN DEPORTIVO;Avda Olímpica 3 A;40.5364952087402;-3.63274455070496;B1;http://www.crtm.es;0;;Europe/Madrid;
8 par_8_06755;06755;MARQUESA VIUDA DE ALDAMA-N*RA.DEL PILAR;Calle de la Marquesa Viuda de Aldama 14;40.5413055419922;-3.6329433917999;
9 par_8_06756;06756;MARQUÉS VALDAVIA-RAMÓN FDEZ.GUISASOLA;Calle del Marqués de la Valdavia 20;40.5408248901367;-3.6367175579071;B1;http://w
10 par_8_06758;06758;MARQUÉS VALDAVIA-NEMESIO CASTRO;Calle del Marqués de la Valdavia 50;40.5429191589355;-3.63905215263367;B1;http://w
11 par_8_06759;06759;MARQUÉS VALDAVIA-AYUNTAMIENTO;Calle del Marqués de la Valdavia 88;40.5447883605957;-3.64296698570251;B1;http://www.cr
12 par_8_06762;06762;AV.ESPAÑA-AYUNTAMIENTO;Avda de España SN;40.5472755432129;-3.6412193775177;B1;http://www.crtm.es;0;;Europe/Madrid;
13 par_8_06763;06763;AV.ESPAÑA-CONCESIONARIO;Avda de España 34;40.5471420288086;-3.64091086387634;B1;http://www.crtm.es;0;;Europe/Madrid;
14 par_8_06766;06766;MARQUÉS VALDAVIA-AYUNTAMIENTO;Calle del Marqués de la Valdavia 69;40.5445098876953;-3.64283442497253;B1;http://www.cr
15 par_8_06767;06767;MARQUÉS VALDAVIA-AYUNTAMIENTO;Calle del Marqués de la Valdavia 37;40.5427932739258;-3.63915705680847;B1;http://www
16 par_8_06768;06768;LIBERTAD-JULIÁN BAENA CASTRO;Calle de la Libertad 26;40.5395812988281;-3.63649344444275;B1;http://www.crtm.es;0;;E
17 par_8_06863;06863;CTRA.AL-CUESTA BLANCA;Ctra A-1 (VIA DE SERVICIO) 14.200;40.5223350524902;-3.65168905258179;B1;http://www.crtm.es;
18 par_8_06864;06864;CTRA.AL-EL ENCINAR DE LOS REYES;Ctra de Irún 13.000;40.5145568847656;-3.65547370910645;B1;http://www.crtm.es;0;;E
19 par_8_06865;06865;AV.BURGOS-DOMINICOS;Avda de Santo Domingo de la Calzada 1;40.5019798278809;-3.66060304641724;A;http://www.crtm.es;
20 par_8_07236;07236;AV.ESPAÑA-M.SEBASTIÁN IZUEL;Avda de España 72;40.5409965515137;-3.62847685813904;B1;http://www.crtm.es;0;;Europe/M
21 par_8_10550;10550;P*LA CASTELLANA-HOSPITAL LA PAZ;Paseo de la Castellana 296;40.4795417785645;-3.68591380119324;A;http://www.crtm.es
```

Comenzamos creando el complemento desde Plugin Builder, rellenamos los datos habituales, de forma similar a como lo hicimos en el primer plugin de ejemplo:

QGIS Plugin Builder - 3.2.1

QGIS Plugin Builder

Class name:

Plugin name:

Description:

Module name:

Version number:

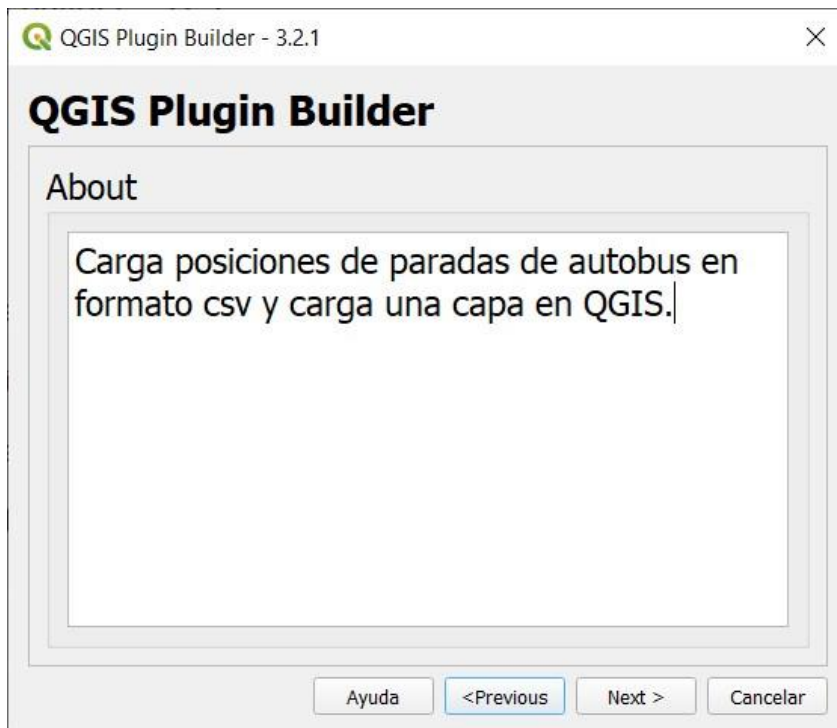
Minimum QGIS version:

Author/Company:

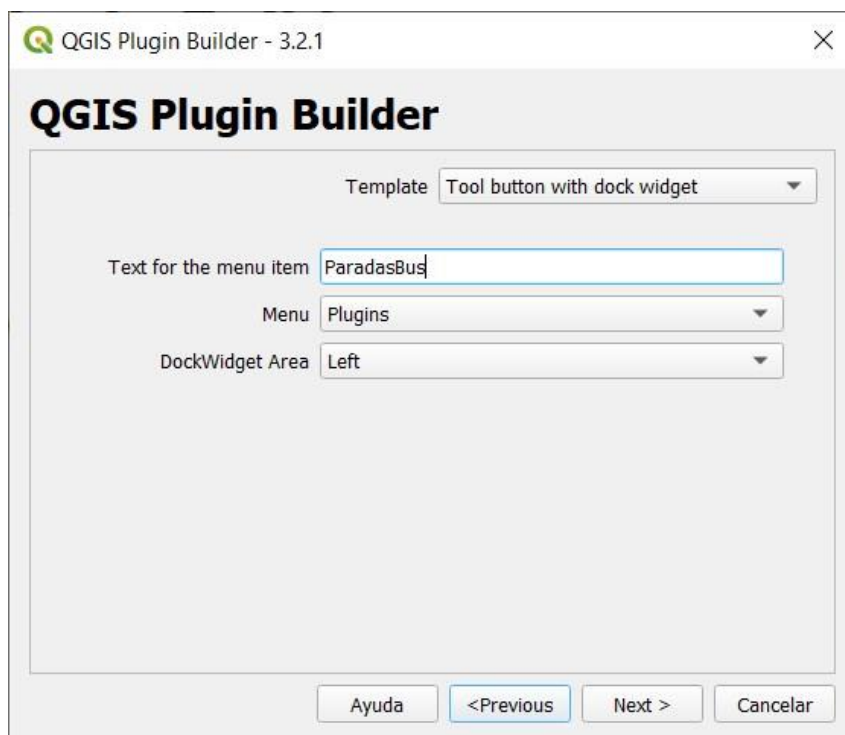
Email address:

Ayuda <Previous Next > Cancelar

Pulsamos siguiente y en el cuadro de diálogo que aparece escribimos una descripción del complemento.



En el siguiente paso seleccionamos como plantilla para la interfaz gráfica del plugin un panel, y escribimos un texto para que se muestre en el menú.



Marcamos las secciones que queremos que aparezcan al crear el complemento.

QGIS Plugin Builder - 3.2.1

QGIS Plugin Builder

- ☒ Internationalization
- ☒ Help
- ☒ Unit tests
- ☒ Helper scripts
- ☐ Makefile
- ☐ pb_tool

Ayuda <Previous Next > Cancelar

Marcamos el plugin como experimental y podemos poner algunas etiquetas.

QGIS Plugin Builder - 3.2.1

QGIS Plugin Builder

Publication (mandatory Items)

Bug tracker

Repository

Publication (recommended Items)

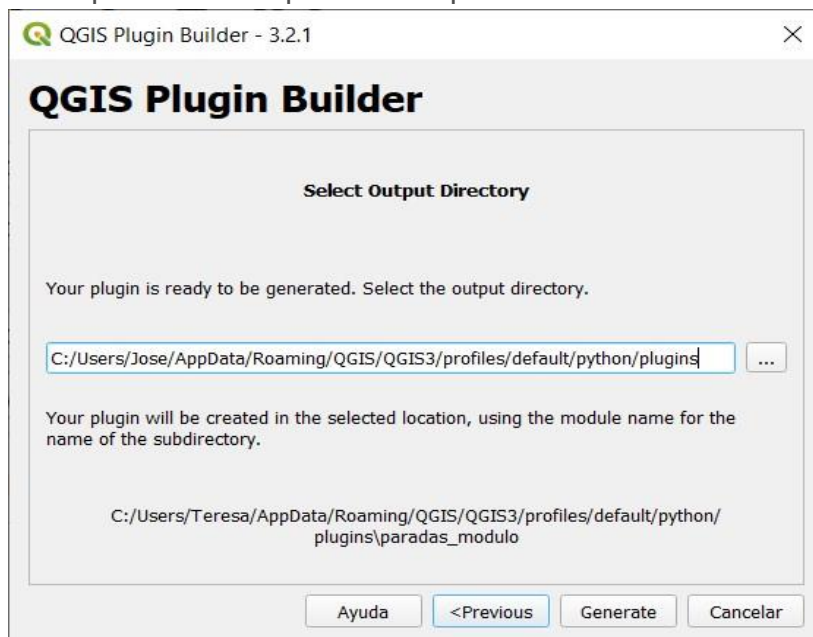
Home page

Tags ...

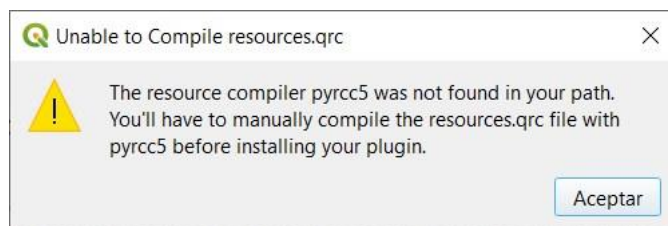
☒ Flag the plugin as experimental

Ayuda <Previous Next > Cancelar

Elegimos la carpeta de salida para el complemento.

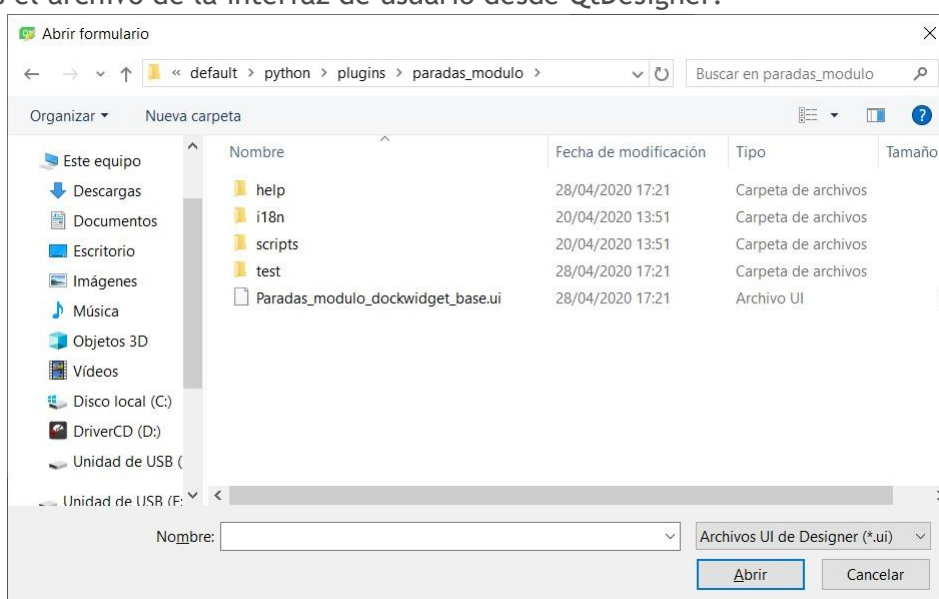


Nos crea el plugin, puede que nos indique que no ha podido compilar los recursos, en ese caso tendremos que compilar los recursos nosotros después.



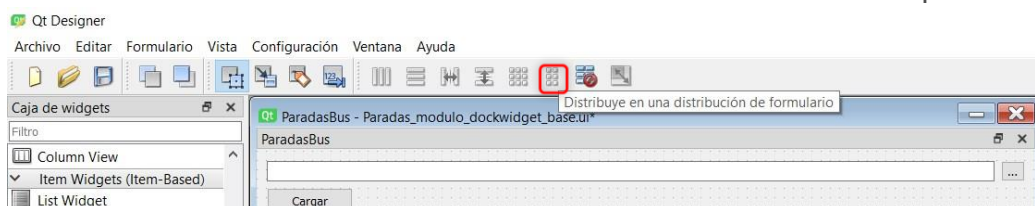
4.1.1 Modificar el diseño del plugin

Abrimos el archivo de la interfaz de usuario desde QtDesigner.



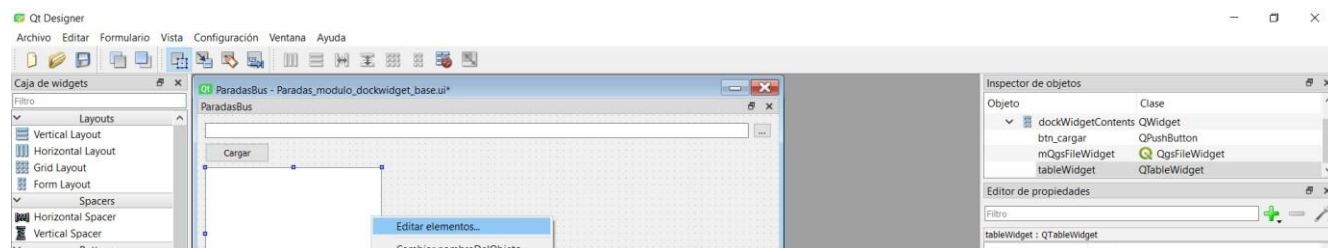
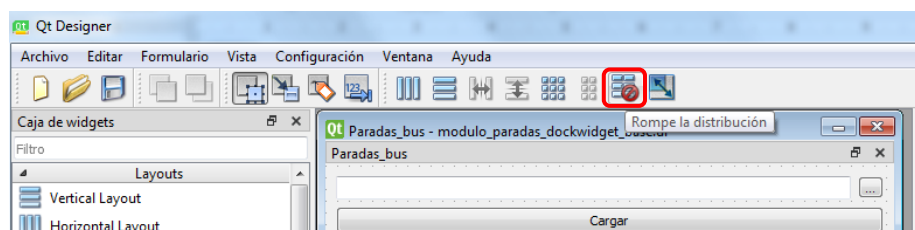
Seleccionamos el archivo *Paradas_modulo_dockwidget_base.ui*.

Para poder colocar los controles como si fuese un formulario activamos la opción:

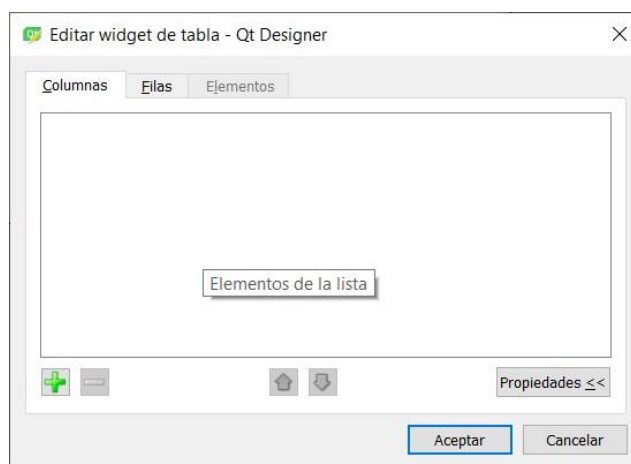


Vamos a colocar un selector de ficheros *QgsFileWidget* con nombre *qfw_selector*, ponemos un *PushButton* con texto *Cargar* y nombre *btn_cargar*, y ponemos una tabla *QTableWidget* con nombre *qtw_tabla*. Tenemos que fijarnos en los nombres para poder hacer referencia a los controles desde el código.

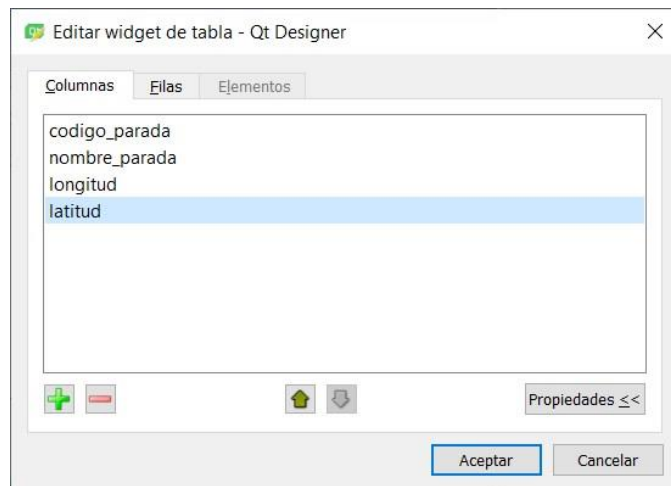
Además si al colocar el botón Cargar se expande ajustándose al ancho del panel, podemos pulsar, con el fondo del panel seleccionado, el botón de romper la distribución.



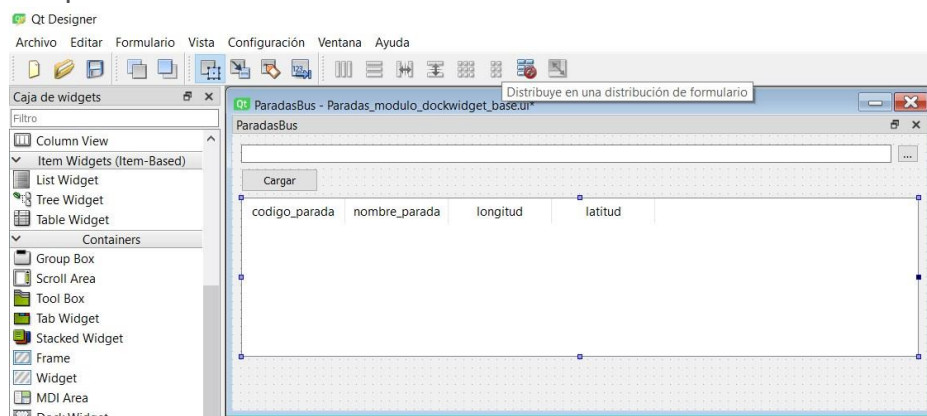
Vamos a pulsar con el botón derecho del ratón sobre la tabla y en *Editar elementos* nos aparece:



Pulsando en el + añadimos títulos para las columnas de la tabla. Ponemos las siguientes columnas:



La tabla queda:



Ya tenemos el diseño creado, lo guardo.

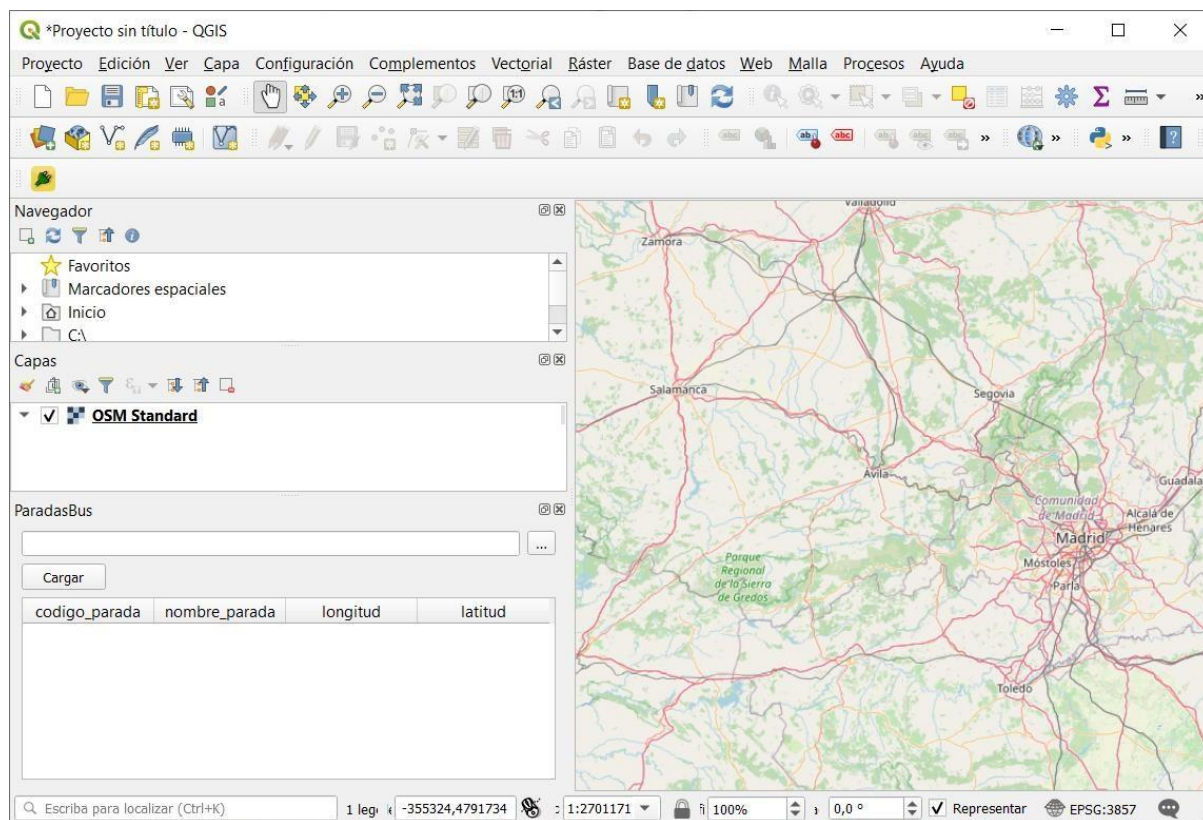
Vamos a la carpeta del plugin y creamos el archivo bat para compilar los recursos:

```
call "C:\Program Files\QGIS 3.10\bin\o4w_env.bat" call
"C:\Program Files\QGIS 3.10\bin\qt5_env.bat" call
"C:\Program Files\QGIS 3.10\bin\py3_env.bat"

set PATH=%PATH%;C:\Program Files\QGIS 3.10\apps\Python37\Scripts\
pyrcc5 -o resources.py resources.qrc
```

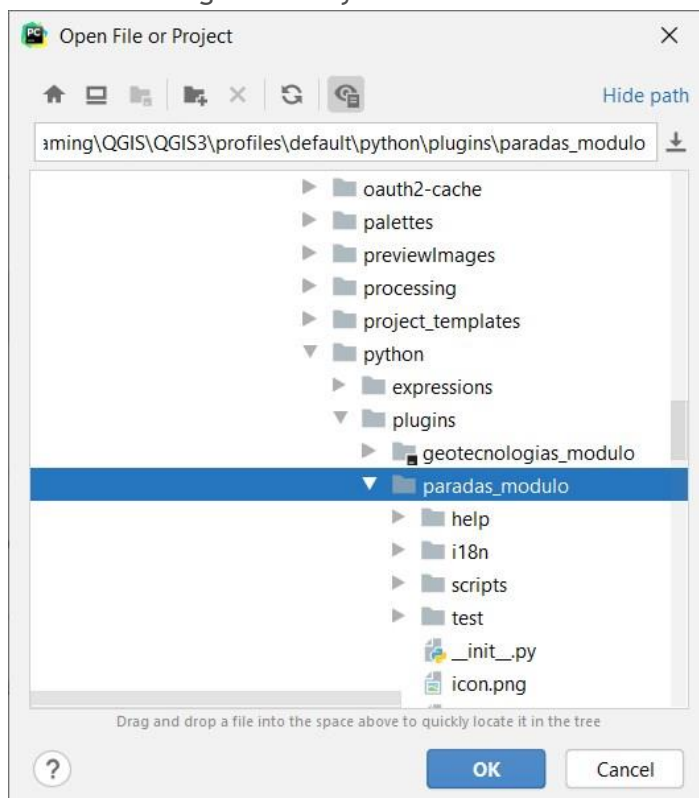
Nos aseguramos que las rutas se corresponden con las correctas de instalación de QGIS. Guardamos el archivo en la carpeta del plugin, y lo ejecutamos haciendo doble clic sobre él.

Probamos a cargar el plugin en QGIS y vemos que se muestra la interfaz, aunque no tiene funcionalidad:



4.1.2 Asignar código al plugin

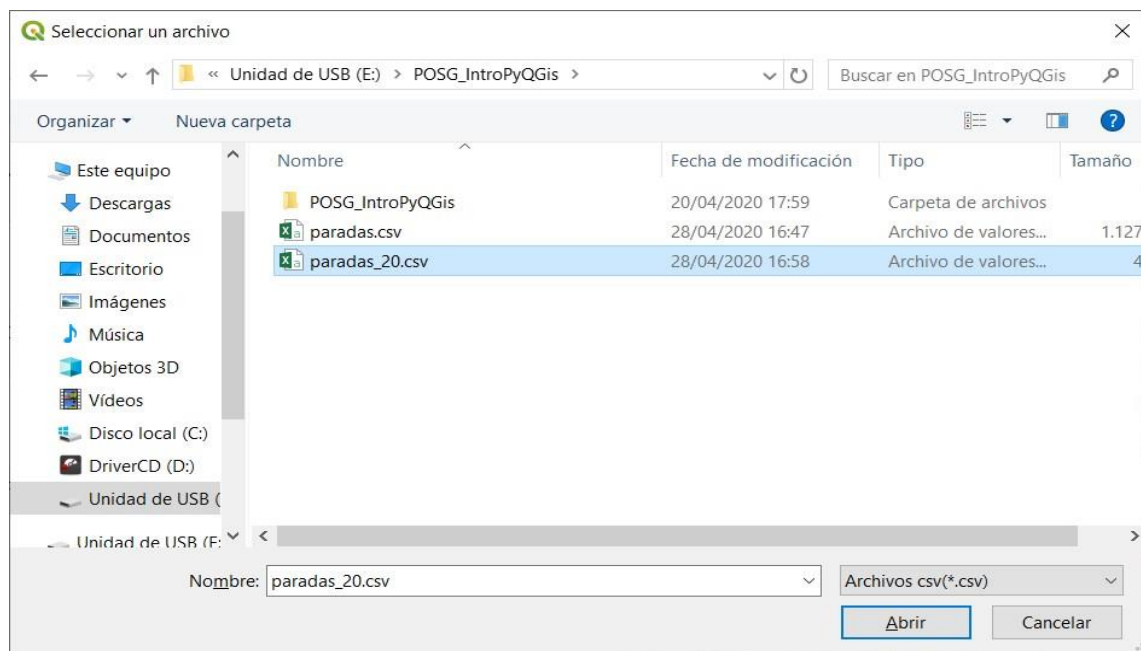
Abrimos el archivo del código desde PyCharm:



Vamos a colocar un filtro para que el selector de archivos solo permita elegir archivos con extensión **.csv**.

```
def __init__(self, parent=None):
    """Constructor."""
    super(ParadasBusDockWidget, self).__init__(parent)
    # Set up the user interface from Designer.
    # After setupUI you can access any designer object by doing
    # self.<objectname>, and you can use autoconnect slots - see
    # http://doc.qt.io/qt-5/designer-using-a-ui-file.html
    # #widgets-and-dialogs-with-auto-connect
    self.setupUi(self)
    self.qfw_selector.setFilter("Archivos csv(*.csv)")
```

Podemos recargar el plugin en QGIS y así vamos comprobando como cambia su funcionalidad:



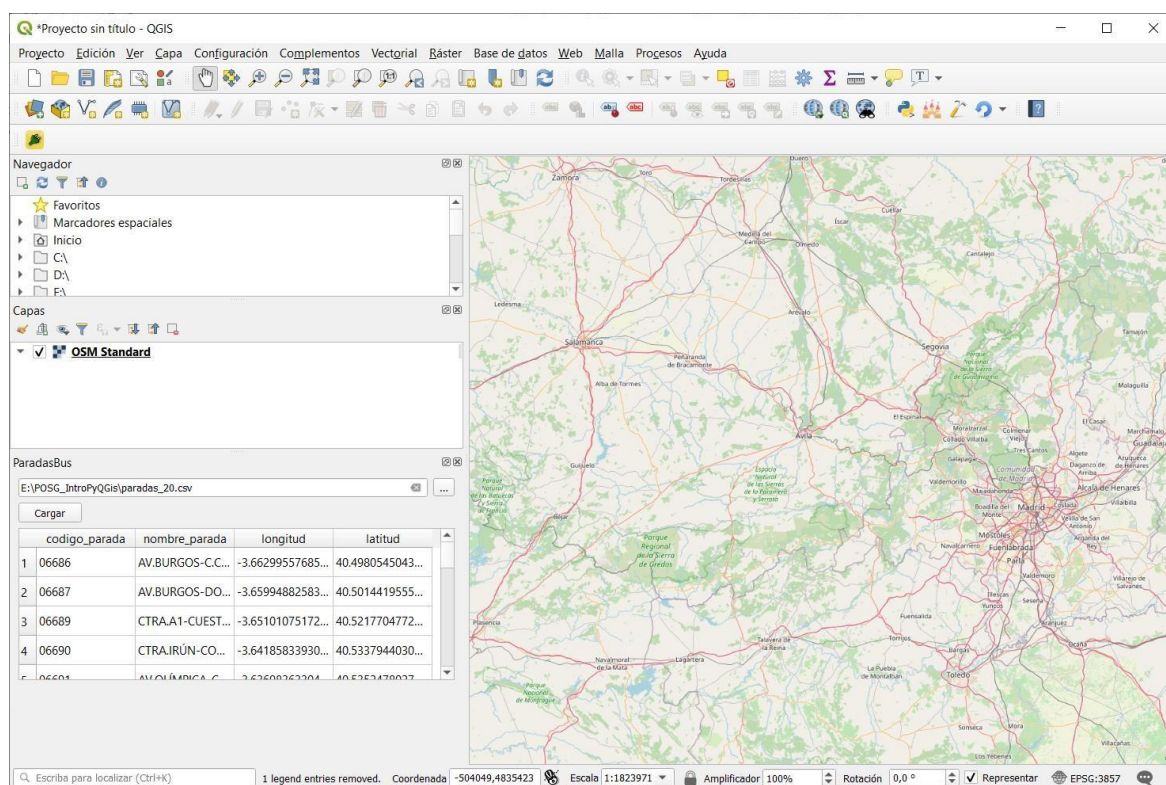
Vamos a asignar código al botón cargar, y creamos una función cargar_datos el archivo con el constructor nos queda:

```
def __init__(self, parent=None):
    """Constructor."""
    super(ParadasBusDockWidget, self).__init__(parent)
    # Set up the user interface from Designer.
    # After setupUI you can access any designer object by doing
    # self.<objectname>, and you can use autoconnect slots - see
    # http://doc.qt.io/qt-5/designer-using-a-ui-file.html
    # #widgets-and-dialogs-with-auto-connect
    self.setupUi(self)
    self.qfw_selector.setFilter("Archivos csv(*.csv)")
    self.btn_cargar.clicked.connect(self.cargar_datos)
```

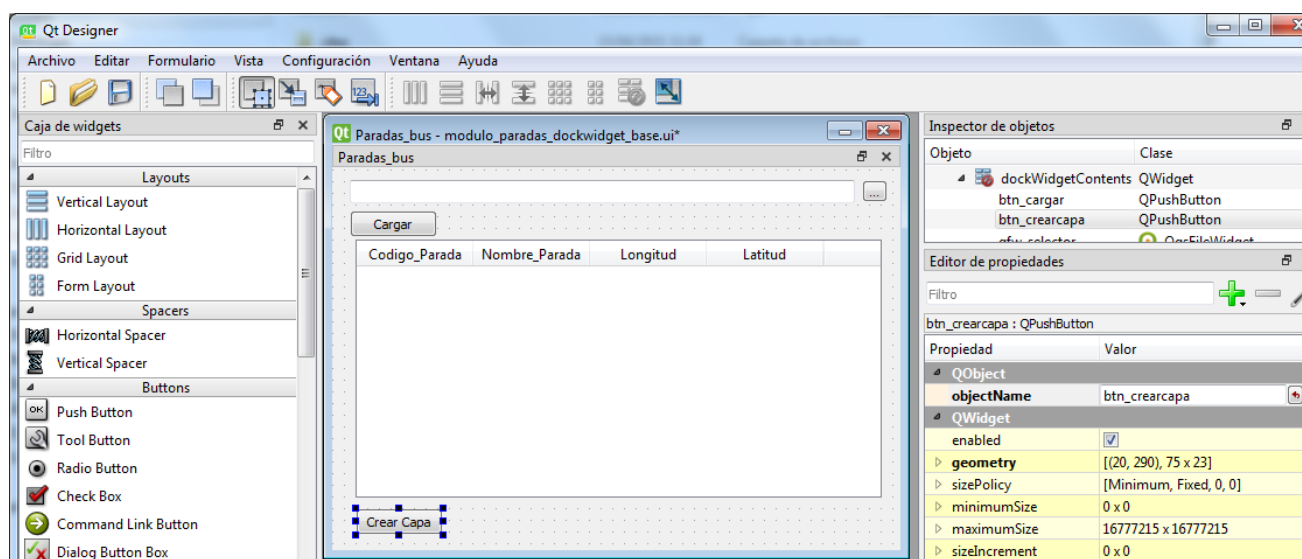
La función quedará:

```
def cargar_datos(self,event):
    ruta=str(self.qfw_selector.filePath())
    with open(ruta,'r') as leer_csv: #abro el archivo para leer
        lineas=leer_csv.read().splitlines()#Separo por lineas
        contar=0
        for linea in lineas:
            if contar>0:
                campos=linea.split(';')
                self.qtw_tabla.insertRow(self.qtw_tabla.rowCount())
                self.qtw_tabla.setItem(self.qtw_tabla.rowCount()-1,0,QtWidgets.QTableWidgetItem((campos[1])))
                self.qtw_tabla.setItem(self.qtw_tabla.rowCount()-1,1,QtWidgets.QTableWidgetItem((campos[2])))
                self.qtw_tabla.setItem(self.qtw_tabla.rowCount()-1,2,QtWidgets.QTableWidgetItem((campos[5])))
                self.qtw_tabla.setItem(self.qtw_tabla.rowCount()-1,3,QtWidgets.QTableWidgetItem((campos[4])))
            contar=contar+1
```

Recargamos el plugin en QGIS, y lo probamos. Al ejecutarlo y pulsar el botón nos aparecerá:



Con el desarrollo que tenemos realizado hasta el momento, somos capaces de cargar el contenido del archivo en formato CSV en la tabla. Ahora vamos a completar la funcionalidad añadiendo otro botón, **Crear Capa**, que nos permita crear una capa en QGIS con los elementos que tenemos en la tabla. Añadimos el botón con el nombre btn_crearcapa.



Volvemos a compilar los recursos, y comprobamos que el archivo resources.py se ha actualizado.

Volvemos a PyCharm y añadimos los imports en el código:

```
import os

from qgis.PyQt import QtGui, QtWidgets, uic
from qgis.PyQt.QtCore import pyqtSignal
from qgis.core import QgsProject, QgsVectorLayer, QgsField, QgsFeature,
QgsGeometry, QgsPointXY
from PyQt5.QtCore import QVariant
```

Después vinculamos el botón con su función:

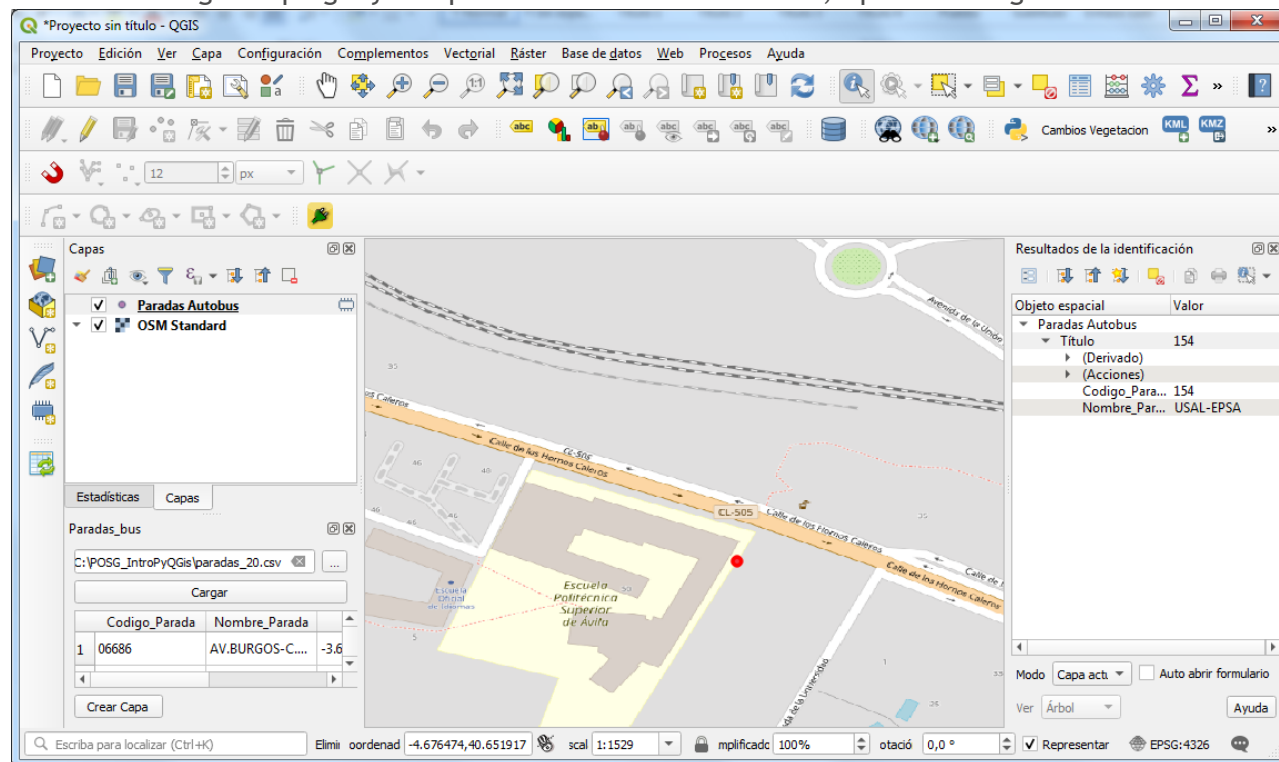
```
self.setupUi(self)
self.qfw_selector.setFilter("Archivos csv(*.csv)")
self.btn_cargar.clicked.connect(self.cargar_datos)
self.btn_crearcapa.clicked.connect(self.crear_capa)
```

Ahora agregamos la función:

```
def crear_capa(self, event):
    vlayer = QgsVectorLayer('Point?crs=EPSG:4326', 'Paradas Autobus', 'memory')
    provider = vlayer.dataProvider()
    provider.addAttributes([QgsField('Codigo_Parada', QVariant.String),
                                QgsField('Nombre_Parada', QVariant.String)])
    vlayer.updateFields()
    f = QgsFeature()
    f.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(-4.678, 40.651)))
    f.setAttributes(['154', 'USAL-EPESA'])
    provider.addFeature(f)
    vlayer.updateExtents()
    QgsProject.instance().addMapLayer(vlayer)
```


Estamos creando una capa nueva en memoria, con nombre “Paradas Bus”, con geometría puntos, y sistema de coordenadas EPSG:4326. Añadimos dos campos de tipo String, con nombres “Codigo Parada” y “Nombre Parada”. Luego añadimos un elemento en la posición de la Escuela Politécnica de Ávila, con esos valores de campo (154, USAL-EPSPA). Actualizamos los datos y añadimos la capa al mapa.

Vamos a recargar el plugin y comprobamos el funcionamiento, Aparecerá algo así:

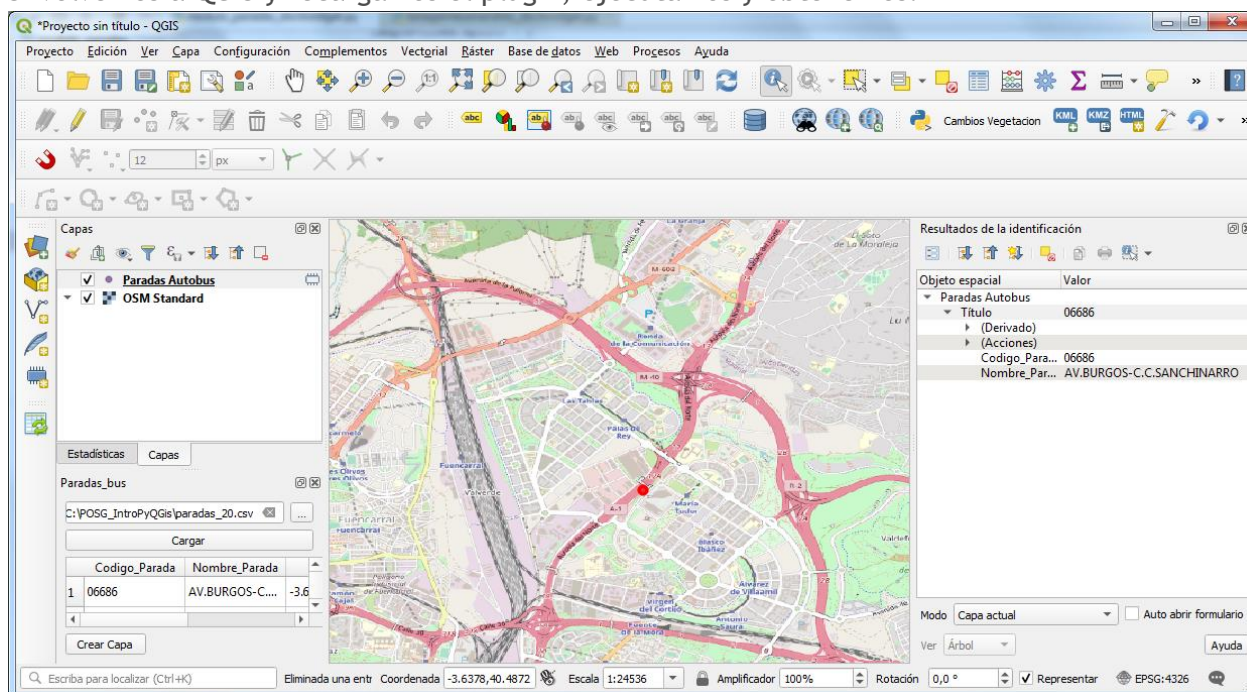


Ahora vamos a intentar modificar el código para que nos cargue los datos de las paradas que tenemos en la tabla. Comenzamos añadiendo solo la parada de la primera fila de la tabla. La primera fila de la tabla será la 0. Las coordenadas de la parada está en las posiciones 2 y 3. El código nos queda:

```
def crear_capa(self,event):
    vlayer = QgsVectorLayer('Point?crs=EPSG:4326', 'Paradas Autobus', 'memory')
    provider = vlayer.dataProvider()
    provider.addAttributes([QgsField('Codigo_Parada', QVariant.String),
                                QgsField('Nombre_Parada', QVariant.String)])

    vlayer.updateFields()
    #Hasta aquí preparo los valores de la capa
    #Introduzco el primer elemento de la tabla
    f = QgsFeature()
    f.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(float(self.qtw_tabla.item(0,2).
    text()),float(self.qtw_tabla.item(0,3).text()))))
    f.setAttributes([self.qtw_tabla.item(0,0).text(),
    self.qtw_tabla.item(0,1).text()])
    provider.addFeature(f)
    #Añado la capa al mapa
    vlayer.updateExtents()
    QgsProject.instance().addMapLayer(vlayer)
```

Si volvemos a QGIS y recargamos el plugin, ejecutamos y obtenemos:



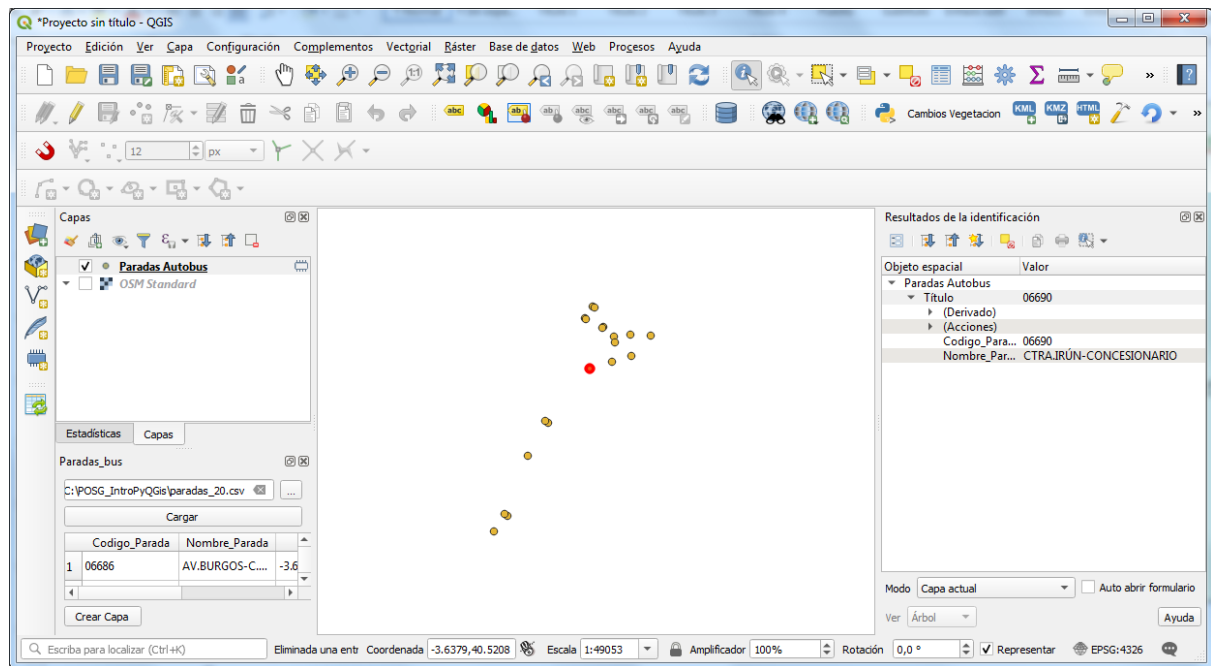
Seguimos añadiendo funcionalidad para recorrer la tabla completa y añadir todas las paradas. Completamos el código:

```
def crear_capa(self, event):
    vlayer = QgsVectorLayer('Point?crs=EPSG:4326', 'Paradas Autobus', 'memory')
    provider = vlayer.dataProvider()
    provider.addAttributes([QgsField('Codigo_Parada', QVariant.String),
                                QgsField('Nombre_Parada', QVariant.String)])

    vlayer.updateFields()
    #Hasta aquí preparo los valores de la capa
    #Introduzco el primer elemento de la tabla
    for fila in range(0, self.qtw_tabla.rowCount() - 1):
        f = QgsFeature()

        f.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(float(self.qtw_tabla.item(fila,
            2).text()), float(self.qtw_tabla.item(fila, 3).text()))))
        f.setAttributes([self.qtw_tabla.item(fila, 0).text(),
            self.qtw_tabla.item(fila, 1).text()])
        provider.addFeature(f)
        #Añado la capa al mapa
        vlayer.updateExtents()
        QgsProject.instance().addMapLayer(vlayer)
```

Ahora volvemos a recargar el plugin y probamos la funcionalidad. Al ejecutarlo nos aparecerán las paradas que teníamos en el archivo .csv.



MÁSTER UNIVERSITARIO
**GEOTECNOLOGÍAS
CARTOGRÁFICAS**

UNIVERSIDAD DE SALAMANCA © 2018



**UNIVERSIDAD
DE SALAMANCA**
CAMPUS DE EXCELENCIA INTERNACIONAL

